

Design, Develop and Implement a menu driven Program in C for the following

operations on **Binary Search Tree (BST)** of Integers

- a. Create a BST of **N** Integers: 6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2
- b. Traverse the BST in Inorder, Preorder and Post Order
- c. Search the BST for a given element (**KEY**) and report the appropriate message
- d. Exit.

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
struct node
```

```
{
```

```
    int info;
```

```
    struct node *llink;
```

```
    struct node *rlink;
```

```
};
```

```
typedef struct node *NODE;
```

```
NODE getnode()
```

```
{
```

```
    NODE x;
```

```
    x=(NODE)malloc(sizeof(struct node));
```

```
    if(x==NULL)
```

```
{  
    printf("Insufficient memory");  
    exit(0);  
}  
return x;  
}
```

```
void preorder(NODE root)  
{  
    if(root==NULL) return;  
    printf("%d\t",root->info);  
    preorder(root->llink);  
    preorder(root->rlink);  
}
```

```
void postorder(NODE root)  
{  
    if(root==NULL) return;  
    postorder(root->llink);  
    postorder(root->rlink);
```

```
    printf("%d\t",root->info);  
}
```

```
void inorder(NODE root)  
{  
    if(root==NULL) return;  
    inorder(root->llink);  
    printf("%d\t",root->info);  
    inorder(root->rlink);  
}
```

```
NODE insert(int item,NODE root)  
{  
    NODE temp,cur,prev;  
    temp=getnode();  
    temp->info=item;  
    temp->llink=NULL;  
    temp->rlink=NULL;  
    if(root==NULL)  
        return temp;  
    prev=NULL;  
    cur=root;  
    while(cur!=NULL)  
    {
```

```

        prev=cur;
    /* if(item==cur->info)
        {
            printf("Duplicate item are not allowed\n");
            free(temp);
            return root;
        }*/
    if(item<cur->info) cur=cur->llink;
    else               cur=cur->rlink;
}
if(item<prev->info)
    prev->llink=temp;
else
    prev->rlink=temp;
return root;
}

```

```

NODE search(int item, NODE root)
{
    NODE cur;
    if(root==NULL) return NULL;
    cur=root;
    while(cur!=NULL)
    {

```

```

        if(item==cur->info) return cur;
        if(item<cur->info)
            cur=cur->llink;
        else
            cur=cur->rlink;
    }
    return NULL;
}

```

```

void main()
{
    int item,ch,n,i;
    NODE root,cur;
    root=NULL;
    while (1)
    {
        printf("\nEnter
        thechoice\n1.Insert\n2.Preorder\n3.Inorder\n4.Postorder\
        n5.Search an Element\n6:Exit\n");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:
                printf("Enter the number of items\n");
                scanf("%d",&n);

```

```
    printf("Enter the item to be inserted\n");
    for(i=0;i<n; i++)
    {
        scanf("%d",&item);
        root=insert(item,root);
    }
    break;
case 2: if(root==NULL)
    {
        printf("Tree is empty\n");
        break;
    }

    printf("Preorder traversal is \n");
    preorder(root);
    printf("\n");
    break;
case 3: if(root==NULL)
    {
        printf("Tree is empty\n");
        break;
    }

    printf("Inorder traversal is \n");
    inorder(root);
```

```

        printf("\n");
        break;
case 4: if(root==NULL)
    {
        printf("Tree is empty\n");
        break;
    }
    printf("Postorder traversal is \n");
    postorder(root);
    printf("\n");
    break;
case 5:     printf("Enter the item to be searched\n");
            scanf("%d",&item);
            cur=search(item,root);
            if(cur==NULL)
                printf("Item not found\n");
            else
                printf("Item found\n");
            break;
case 6: exit(0);
default: printf("Enter valid choice\n");
    }
}
}

```

